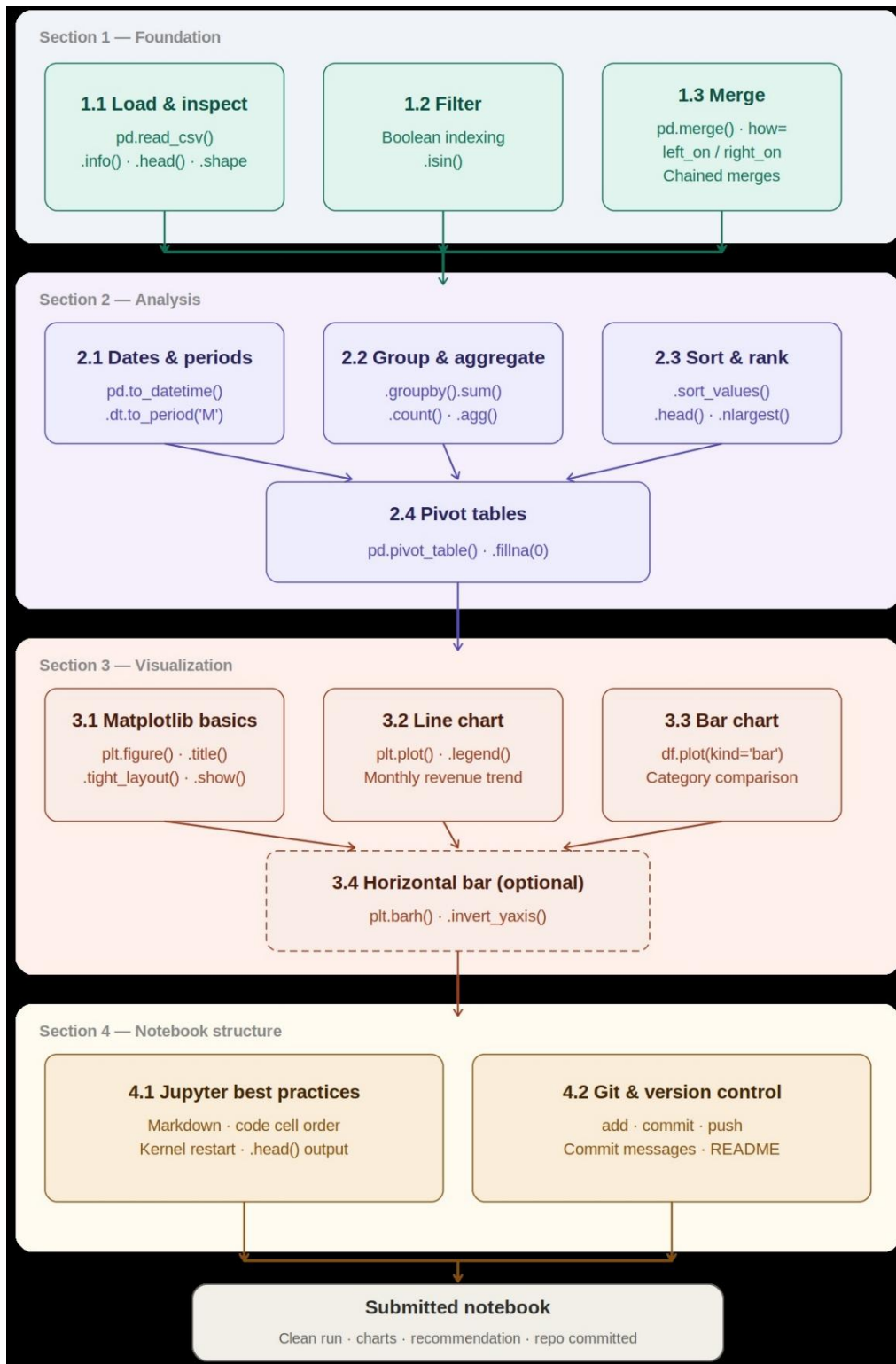


Capstone 2 Skill Check



Suggested Workflow

How to Use This Guide

This guide is a concise reference for the Python and data analysis skills needed in the Capstone 2 Project. For each skill you will find:

- What the function or technique does and why it matters
- The skill level tag: Foundation, Core skill.
- Clickable resource links to official documentation and video walkthroughs

Work through sections in order — earlier skills are prerequisites for later ones. Use the Self-Assessment Checklist on the last page to track your confidence.

SECTION 1 — Foundation Skills

These skills underpin the Foundation Skills. Make sure you are comfortable with all of them before starting the notebook.

1.1 Loading & Inspecting Data with Pandas

`pd.read_csv()`
Foundation

Load each CSV file into a DataFrame. Every analysis starts here. Pass the file path as a string and pandas returns a DataFrame with all rows and columns from the CSV.

`df.info()`
Foundation

Check column names, data types, and non-null counts after loading. This is the first thing to run after any `pd.read_csv()` call to confirm the data loaded correctly.

`df.head()` /
`df.shape`
Foundation

`df.head()` previews the first 5 rows (pass a number for more). `df.shape` returns a tuple (rows, columns). Together they confirm the dimensions of your DataFrame without printing everything.

Resources

Topic / Skill	Resource & Link
<code>pd.read_csv()</code>	Pandas read_csv documentation
<code>df.info()</code> & <code>.head()</code>	Pandas DataFrame inspection guide
Pandas getting started	10 minutes to pandas (official)
Video walkthrough	Corey Schafer — Pandas Tutorial Part 1 (YouTube)

1.2 Filtering DataFrames

Boolean indexing <i>Foundation</i>	Select only the rows that match a condition: <code>df[df['column'] == value]</code> . Returns a new DataFrame. Chain conditions with <code>&</code> (and) or <code> </code> (or), wrapping each condition in parentheses.
<code>.isin()</code> <i>Foundation</i>	Filter rows where a column value matches any item in a list: <code>df[df['col'].isin([val1, val2])]</code> . More readable than chaining multiple <code>==</code> conditions when checking several values at once.

Resources

Topic / Skill	Resource & Link
Boolean indexing	Pandas indexing & selecting data
<code>.isin()</code> method	pandas.Series.isin documentation
Video — filtering	Data School — Filtering rows in pandas (YouTube)

1.3 Merging DataFrames

<code>pd.merge()</code> / <code>df.merge()</code> <i>Core skill</i>	Combine two DataFrames on a shared column: <code>df1.merge(df2, on='SharedColumn')</code> . Returns a new DataFrame with columns from both. Used in every Step 6 question to join sales, location, product, and customer data.
<code>how='left'</code> <i>Core skill</i>	Keep every row from the left DataFrame even when there is no matching row on the right. Rows without a match get NaN in the right-side columns. Critical for Q4 where not every transaction has a rewards member.
<code>left_on</code> / <code>right_on</code> <i>Core skill</i>	Use when the join columns have different names in each DataFrame. Example: <code>left_on='RewardsID'</code> , <code>right_on='cust_id'</code> links the transactions file to the members file even though the column names differ.
Chaining multiple merges <i>Core skill</i>	Combine three or more files by chaining <code>.merge()</code> calls: <code>df1.merge(df2, on='A').merge(df3, on='B')</code> . Required for Q5 which links transactions → inventory → categories to get readable category names.

Resources

Topic / Skill	Resource & Link
<code>pd.merge()</code>	Pandas merge documentation
Join types explained	Pandas merge, join, concatenate guide
Video — merging	Corey Schafer — Merging DataFrames (YouTube)
Visual join guide	SQL joins visualized (applies to pandas)

SECTION 2 — Analysis Skills

2.1 Working with Dates & Time Periods

`pd.to_datetime()`
Core skill

Convert a column of date strings (dtype object) into proper datetime objects. Required before you can extract months or years. Example: `df['TransactionDate'] = pd.to_datetime(df['TransactionDate'])`.

`.dt.to_period('M')`
Core skill

Collapse a full datetime value into a year-month label such as 2024-01. The result is a Period object that sorts chronologically and displays cleanly on chart axes. Assign it to a new YearMonth column.

Resources

Topic / Skill	Resource & Link
<code>pd.to_datetime()</code>	Pandas to_datetime docs
Time series in pandas	Pandas time series / date guide
<code>.dt</code> accessor	DatetimeIndex properties (.dt)

2.2 Grouping & Aggregating

`.groupby().sum()`
Core skill

Group rows by one or more columns and total a numeric column: `df.groupby('StoreID')['SaleAmount'].sum()`. Used for monthly revenue (Q2), store ranking (Q3), top customers (Q4), and category revenue (Q5).

`.groupby().count()`
`/ .size()`
Core skill

Count the number of rows per group. `.count()` counts non-null values in a specific column; `.size()` counts all rows including nulls. Required for Q5 to find the number of transactions per category per month.

Grouping by
multiple columns
Core skill

Pass a list to `groupby` to break down across two dimensions at once: `df.groupby(['YearMonth', 'CategoryName'])['SaleAmount'].sum()`. The result has a MultiIndex; use `.reset_index()` to flatten it.

`.reset_index()`
Core skill

After a `groupby` operation, the group keys become the index. `reset_index()` turns them back into regular columns, making the DataFrame easier to display, sort, and pass into a chart.

`.agg()`
Core skill

Apply multiple aggregation functions at once: `df.groupby('StoreID').agg({'SaleAmount': ['sum', 'mean', 'count']})`. Returns a DataFrame with one column per aggregation, useful for summary tables.

Resources

Topic / Skill	Resource & Link
GroupBy guide	Pandas groupby: split-apply-combine
.agg() for multiple ops	Pandas aggregation functions
Video — groupby	Corey Schafer — GroupBy (YouTube)

2.3 Sorting & Ranking

<code>.sort_values()</code> <i>Foundation</i>	Order rows by one or more columns: <code>df.sort_values('SaleAmount', ascending=False)</code> puts the highest values first. Pass a list to sort by multiple columns. Use <code>ascending=True</code> for lowest-first ranking.
<code>.head(n)</code> / <code>.tail(n)</code> <i>Foundation</i>	<code>.head(n)</code> returns the first n rows; <code>.tail(n)</code> returns the last n. After sorting descending, <code>.head(10)</code> gives the top 10 performers. Default n is 5 when no argument is passed.
<code>.nlargest()</code> / <code>.nsmallest()</code> <i>Foundation</i>	Shortcut for sort + head: <code>df['SaleAmount'].nlargest(10)</code> returns the 10 highest values directly. More concise than <code>.sort_values(ascending=False).head(10)</code> and works on a Series or DataFrame column.

Resources

Topic / Skill	Resource & Link
<code>.sort_values()</code>	Pandas sort values docs
Ranking options	Pandas rank method
<code>.nlargest()</code>	pandas.DataFrame.nlargest

2.4 Pivot Tables

<code>pd.pivot_table()</code> <i>Step 8 skill</i>	Reshape a DataFrame so that unique values in one column become new column headers. Syntax: <code>pd.pivot_table(df, index='CategoryName', columns='TerritoryID', values='SaleAmount', aggfunc='sum')</code> . The result is the direct input for Chart B.
<code>.fillna(0)</code> <i>Step 8 skill</i>	Pivot tables produce NaN wherever a combination has no data. Chain <code>.fillna(0)</code> after <code>pivot_table()</code> to replace those with zero so the grouped bar chart renders all bars correctly without errors.

Resources

Topic / Skill	Resource & Link
<code>pd.pivot_table()</code>	Pandas pivot table docs

SECTION 3 — Visualization Skills

Matplotlib is the charting library used in this project. You need at least two charts with clear titles, labels, and legends. Each subsection below covers one chart type.

3.1 Matplotlib Basics

```
import
matplotlib.pyplot
as plt
Foundation
```

Standard import alias for Matplotlib. Always import at the top of your notebook with this exact alias. All chart functions are then accessed as `plt.plot()`, `plt.title()`, etc.

```
plt.figure(figsize=(w,
h))
Foundation
```

Create a new canvas before drawing. Width and height are in inches. Use (12, 5) for wide trend charts with many x-axis labels and (10, 6) for bar charts comparing categories. Always call this before `plt.plot()` or `.plot()`.

```
plt.title() /
plt.xlabel() /
plt.ylabel()
Foundation
```

Add a descriptive title and axis labels to every chart. A chart without labels is unreadable to a non-technical audience. Graders will check for these on every chart you submit.

```
plt.tight_layout()
Foundation
```

Automatically adjusts subplot spacing to prevent labels, titles, and tick marks from overlapping or being clipped. Call this just before `plt.show()` on every chart.

```
plt.show()
Foundation
```

Render and display the chart in the notebook output. Without this call the chart may not appear, especially when running all cells at once. Always the last line in a chart code cell.

Resources

Topic / Skill	Resource & Link
Matplotlib basics	Official Matplotlib tutorials
Pyplot API	matplotlib.pyplot reference
Video — basics	Corey Schafer — Matplotlib Tutorial (YouTube)

3.2 Line Charts (e.g. Chart A —Monthly Revenue Trend)

```
plt.plot(x, y,
marker=, label=,
linestyle=)
```

Draw a line chart. `x` is the list of month labels (YearMonth), `y` is the list of revenue values. Use `marker='o'` to show data points. Call `plt.plot()` once per territory on the same figure to overlay two lines.

Core skill	
<pre>plt.xticks(rotation=45)</pre> Core skill	Rotate the x-axis tick labels by 45 degrees to prevent them from overlapping when month labels are long (e.g., '2024-01', '2024-02'). Add ha='right' to right-align labels after rotation for cleaner appearance.
<pre>plt.legend()</pre> Core skill	Display a legend box showing which line corresponds to which territory. Requires the label= argument in each plt.plot() call. Add legend(title='Territory') to include a bold header inside the legend box.
<pre>plt.grid(True, linestyle='--', alpha=0.5)</pre> Core skill	Add a subtle background grid to make it easier to read values off the y-axis. Use linestyle='--' for dashed lines and alpha for transparency so the grid does not compete visually with the data lines.

Resources

Topic / Skill	Resource & Link
plt.plot()	Matplotlib plot docs
Line chart tutorial	Matplotlib — Basic line plot example

3.3 Bar Charts (e.g. Chart B — Category Revenue Comparison)

<pre>df.plot(kind='bar', figsize=, ax=)</pre> Core skill	Plot a grouped bar chart directly from a pivot table DataFrame. Each row becomes a group of bars, one bar per column (territory). Pass figsize=(12, 6) for a readable chart and ax=plt.gca() to use the current figure.
<pre>plt.xticks(rotation=30, ha='right')</pre> Core skill	Rotate category labels on the x-axis 30 degrees and right-align them. 30 degrees is less steep than 45 and often looks cleaner for short category names. Adjust the angle as needed for your category names.
<pre>legend(title=, bbox_to_anchor=)</pre> Core skill	Place the legend outside the chart area when it would otherwise cover bars: legend(title='Territory', bbox_to_anchor=(1.01, 1), loc='upper left'). Remember to add extra right margin with plt.tight_layout() after.

Resources

Topic / Skill	Resource & Link
Bar chart docs	Matplotlib — Bar chart examples
DataFrame.plot()	Pandas built-in plotting reference

3.4 Horizontal Bar Chart (e.g. Chart C — Store Ranking)

<code>plt.barh(y, width)</code> <i>Optional</i>	Draw a horizontal bar chart. <code>y</code> is a list of store names or IDs (the labels on the vertical axis); <code>width</code> is the list of revenue values (the bar lengths). Easier to read than vertical bars when labels are long strings.
<code>plt.gca().invert_yaxis()</code> <i>Optional</i>	Flip the y-axis so the highest-ranked store appears at the top of the chart. By default matplotlib draws bars from bottom to top, which reverses a descending ranking. Call after <code>plt.barh()</code> and before <code>plt.show()</code> .
<code>plt.barh color and edgecolor</code> <i>Optional</i>	Customize bar appearance: <code>plt.barh(y, width, color='steelblue', edgecolor='white')</code> . Use a single color string for uniform bars or pass a list of colors to highlight specific stores (e.g., gold for #1).

Resources

Topic / Skill	Resource & Link
<code>plt.barh()</code>	Matplotlib barh docs
Horizontal bar example	Matplotlib — Horizontal bar chart

SECTION 4 — Notebook Structure Skills

4.1 Jupyter Notebook Best Practices

Markdown cells <i>Foundation</i>	Write plain-English explanations in Markdown cells before each code cell (introduce the question) and after (interpret the result). Use <code>##</code> for section headings, <code>**text**</code> for bold, and regular prose for narrative.
Code cell organisation <i>Foundation</i>	One logical operation per code cell: one <code>pd.read_csv()</code> , one merge, one groupby. Avoid packing several unrelated steps into one cell. This makes debugging easier and keeps the notebook readable as a report.
Kernel Restart + Run All <i>Critical</i>	Before submitting, use Kernel → Restart & Run All to execute every cell from scratch in order. The notebook must complete without errors. Cells that rely on variables defined in a later cell will fail this test.
Displaying results cleanly <i>Good practice</i>	End a code cell with the DataFrame variable name (not <code>print()</code>) so Jupyter renders it as a formatted table. Use <code>.head(10)</code> or <code>.head(20)</code> to cap the output length — never display hundreds of rows mid-notebook.

Interpretation Markdown cells <i>Foundation</i>	After every analysis result, add a Markdown cell that answers the 'so what?': name the top result, give its value, and state what it implies for the marketing team. This is what separates a data story from a data dump.
---	--

Resources

Topic / Skill	Resource & Link
Jupyter Markdown guide	Jupyter Notebook Markdown basics
Markdown syntax	Markdown Guide — basic syntax
Notebook best practices	Towards Data Science — Jupyter tips

4.2 Git & Version Control

git add / git commit / git push <i>Required</i>	Stage changed files (git add .), create a snapshot with a message (git commit -m 'message'), and upload to GitHub (git push). Commit at least once per day so reviewers can see your progress over time.
---	--

Meaningful commit messages <i>Required</i>	Write messages that describe the change, not just that something changed. 'Complete Q2 monthly revenue analysis' is useful. 'update', 'fix', or 'wip' tell a reviewer nothing. Use present-tense imperative style.
--	--

README.md <i>Required</i>	A short README in the repository root describes the project, the territories analyzed, and how to run the notebook. Include the link to your video recording here. Reviewers read the README before opening any file.
-------------------------------------	---

Resources

Topic / Skill	Resource & Link
Git basics	Git — Getting Started guide
Git commit best practices	Conventional commits guide
GitHub README guide	GitHub — About READMEs

SECTION 5 — Self-Assessment Checklist

Check off each skill when you feel confident using it independently

✓	Skill
☐	Load CSV files with <code>pd.read_csv()</code> and inspect with <code>.info()</code>
☐	Filter rows with boolean indexing and <code>.isin()</code>
☐	Merge two DataFrames on a shared column with <code>pd.merge()</code>
☐	Perform a left merge using <code>left_on</code> / <code>right_on</code>
☐	Chain multiple merges to combine 3+ DataFrames
☐	Convert a date column to datetime with <code>pd.to_datetime()</code>
☐	Create a YearMonth period column with <code>.dt.to_period('M')</code>
☐	Group by one column and aggregate with <code>.sum()</code> or <code>.count()</code>
☐	Group by multiple columns at once and reset the index
☐	Sort results with <code>.sort_values()</code> and select top rows with <code>.head()</code>
☐	Create a pivot table with <code>pd.pivot_table()</code> and fill NaN with <code>.fillna(0)</code>
☐	Create a line chart with <code>plt.plot()</code> — with labels, legend, and grid
☐	Create a grouped bar chart from a pivot table using <code>df.plot(kind='bar')</code>
☐	Rotate x-axis labels with <code>plt.xticks(rotation=...)</code>
☐	Create a horizontal bar chart and invert the y-axis (optional)
☐	Write Markdown cells to introduce and interpret each question
☐	Structure notebook: markdown → code → interpretation per question
☐	Use <code>plt.figure()</code> , <code>plt.tight_layout()</code> , and <code>plt.show()</code> on every chart
☐	Restart kernel and run all cells without errors before submitting
☐	Commit changes daily with meaningful commit messages
☐	Write a recommendation citing specific data analysis results